

Verteilte Systeme, Containerisierung

Arbeitsblatt 06 · Helm, Operator und CloudNativePG

Vorwort

Grundlage für diese Aufgabe ist die Präsentation «06_Helm_Operator_CloudNativePG». Sie erweitern den funktionierenden Stand aus Block 5 um einen hoch verfügbaren PostgreSQL-Cluster.

Arbeitszeit	Besprechung	Total
110 Minuten	20 Minuten	130 Minuten

Lernziel

Sie können Helm, Kustomize und einen Operator sauber voneinander abgrenzen, persistente Daten über einen stabilen Service anbinden und einen echten Rollenwechsel beobachten.

Material

- Ihr funktionierender Projektstand nach Arbeitsblatt 05
- Block 6: <https://github.com/SwitzerChees/vsc-dispatch-city-06-persistence> (Release v1.1.0 nach Review)
- Docker Desktop, k3d, kubectl, Helm, Git und Browser
- macOS/Linux/WSL/Git Bash oder Windows PowerShell

Wichtig: Arbeiten Sie immer im eigenen Projektordner. Das Block-6-Paket ergänzt den Stand, ersetzt aber nicht Ihr Dashboard.

Der neue Anspruch

Nicht nur ausführen	Sondern auch entscheiden
Ressourcen rendern	Wer verwaltet welches Objekt?
Status prüfen	Welche Adresse bleibt stabil?
Ausfall auslösen	RTO und Datenbestand beurteilen

Sichtbares Resultat: Am Ende läuft das visuelle System weiter, schreibt in PostgreSQL und übersteht einen Primary-Ausfall.

1

Baustein integrieren und Wunschzustand lesen

Erst verstehen, dann anwenden.

Integrieren Sie das Paket in Ihren Block-5-Stand. Wenden Sie noch nichts auf Kubernetes an.

- Führen Sie das passende Installationsskript aus.
- Rendern Sie das neue Overlay lokal.
- Lesen Sie danach die Cluster-Resource und beantworten Sie die Architekturfragen.

INTEGRATION UND RENDERING

```
# Bash / macOS / Linux / WSL / Git Bash
../vsc-dispatch-city-06-persistence/install.sh .
kubectl customize deploy/overlays/block-06-persistence > /tmp/block6.yaml

# Windows PowerShell
& '..\vsc-dispatch-city-06-persistence\install.ps1' -Target '.'
kubectl customize deploy/overlays/block-06-persistence | Out-File block6.yaml
```

Frage	Ihre Prognose vor dem Deployment
Wie viele DB-Instanzen?	2
Welches PostgreSQL-Image?	ghcr.io/cloudnative-pg/postgresql:18.4-system-trixie
Welcher Service nimmt Writes an?	food-delivery-db-rw
Wer erzeugt die DB-Pods?	der CloudNativePG Operator

Sichtbares Resultat: Das Overlay rendert ohne Fehler und Ihre vier Prognosen sind aus YAML begründet.

Begründen Sie eine Prognose mit Resource-Name und Feldpfad:

Cluster food-delivery-db, spec.instances: 2
-rw kommt aus Secret database-url, den Service macht der Operator

2

Chart prüfen und Operator als Release installieren

Helm nicht als Blackbox verwenden.

Prüfen Sie zuerst Metadaten und Kurs-Values. Installieren Sie danach den Operator.

CHART UNTERSUCHEN

```
helm repo add cnpg https://cloudnative-pg.github.io/charts
helm repo update
helm show chart cnpg/cloudnative-pg --version 0.29.0
helm show values cnpg/cloudnative-pg --version 0.29.0
```

Öffnen Sie zusätzlich platform/cloudnative-pg/values-course.yaml. Entscheiden Sie, welche Werte Chart-Defaults sind und welche wir bewusst überschreiben.

Beobachtung	Default oder Kurs-Override?	Warum sinnvoll?
Chart-Version 0.29.0	kein Default, Kurs pinnt per --version	alle haben die gleiche Version
Operator-Ressourcen	Override, 50m / 96Mi	reicht für k3d lokal
Monitoring deaktiviert	Default	noch kein Prometheus da

OPERATOR INSTALLIEREN

```
# Bash
./platform/cloudnative-pg/install.sh

# PowerShell
./platform/cloudnative-pg/install.ps1

helm -n cnpg-system list
kubectl get crd clusters.postgresql.cnpg.io
```

Sichtbares Resultat: Das Helm-Release ist deployed und die Kubernetes-API kennt die Resource-Art Cluster.

Formulieren Sie in einem Satz: Was installiert Helm, und was betreibt der Operator?

Helm installiert den Operator und die CRDs, der Operator betreibt dann die DB.

3

Datenbank deployen und Service-Routing erklären

Generierte Objekte statt nur Pod-Status prüfen.

- Bauen und importieren Sie die Images mit dem passenden Skript.
- Wenden Sie das Block-6-Overlay an und warten Sie auf zwei Instanzen.
- Ordnen Sie danach die drei Services ihren aktuellen Endpoints zu.

DEPLOYMENT

```
# Bash
./scripts/build-images.sh
CLUSTER=teko-k8s ./scripts/load-images.sh

# PowerShell
./scripts/build-images.ps1
./scripts/load-images.ps1 -Cluster teko-k8s

kubectl --context k3d-teko-k8s apply -k deploy/overlays/block-06-persistence
kubectl --context k3d-teko-k8s -n food-delivery wait --for=condition=Ready cluster/food-delivery-db --
timeout=5m
```

ROUTING PRÜFEN

```
kubectl --context k3d-teko-k8s -n food-delivery get pods -L cnpg.io/instanceRole
kubectl --context k3d-teko-k8s -n food-delivery get svc food-delivery-db-rw food-delivery-db-ro food-
delivery-db-r
kubectl --context k3d-teko-k8s -n food-delivery get endpointslice -l kubernetes.io/service-name=food-
delivery-db-rw -o wide
```

Service	Aktuelle Zielrolle	Geeignet für
food-delivery-db-rw	Primary	schreiben
food-delivery-db-ro	Standby	lesen
food-delivery-db-r	alle	beliebige Reads

Sichtbares Resultat: Zwei DB-Pods sind Ready; -rw zeigt genau auf den aktuellen Primary.

Warum darf der Order Worker keinen Pod-Namen als Host verwenden?

Pod-Name ändert sich beim Failover, -rw zeigt immer auf den aktuellen Primary

4

Persistenz sichtbar nachweisen

Neustart ist kein Datenverlust-Test ohne vorherigen Zustand.

Erzeugen Sie im Dashboard eine klar erkennbare Bestellung und notieren Sie deren ID. Starten Sie anschliessend nur die zustandsführenden Anwendungen neu.

TERMINAL A

```
kubectl --context k3d-teko-k8s -n food-delivery port-forward service/traefik 8080:80
# Browser: http://localhost:8080
```

Bestell-ID und sichtbarer Status vor dem Neustart:

559aaecd-26ba-4eff-b774-3f52de9b7195

delivered

TERMINAL B

```
kubectl --context k3d-teko-k8s -n food-delivery rollout restart deployment/order-worker
deployment/control-api
kubectl --context k3d-teko-k8s -n food-delivery rollout status deployment/order-worker
kubectl --context k3d-teko-k8s -n food-delivery rollout status deployment/control-api
```

Laden Sie das Dashboard neu und suchen Sie dieselbe Bestellung. Prüfen Sie zusätzlich direkt in PostgreSQL:

DIREKT IN POSTGRESQL PRÜFEN

```
# Bash
PRIMARY=$(kubectl --context k3d-teko-k8s -n food-delivery get pod -l cnpg.io/cluster=food-delivery-db,cnpg.io/instanceRole=primary -o jsonpath='{.items[0].metadata.name}')
kubectl --context k3d-teko-k8s -n food-delivery exec "$PRIMARY" -- psql -d delivery -c 'SELECT id,status FROM orders ORDER BY updated_at DESC LIMIT 5;'

# PowerShell
$Primary = kubectl --context k3d-teko-k8s -n food-delivery get pod -l cnpg.io/cluster=food-delivery-db,cnpg.io/instanceRole=primary -o jsonpath='{.items[0].metadata.name}'
kubectl --context k3d-teko-k8s -n food-delivery exec $Primary -- psql -d delivery -c 'SELECT id,status FROM orders ORDER BY updated_at DESC LIMIT 5;'
```

Sichtbares Resultat: Die Bestellung bleibt nach beiden Rollouts im Dashboard und in PostgreSQL erhalten.

Welche Schicht sorgt hier für Persistenz, welche für die Rekonstruktion der Anzeige?

Persistenz: PostgreSQL auf dem PVC

Anzeige: control-api liest aus der DB, Dashboard zeigt es an

5

Doppelte Nachricht, einfache Wirkung

Idempotenz mit Datenbankevidenz belegen.

Das Lab publiziert dieselbe event_id zweimal. Bevor Sie es starten: Sagen Sie voraus, welche Zähler nach korrekter Verarbeitung den Wert 1 haben müssen.

Messpunkt	Prognose	Begründung
processed_events für EVENT_ID	1	event_id ist beim 2. Mal schon drin
orders für ORDER_ID	1	Duplikat wird übersprungen, Order nur einmal

TEST AUSFÜHREN UND EVENT_ID / ORDER_ID NOTIEREN

```
kubect1 --context k3d-teko-k8s -n food-delivery port-forward service/rabbitmq 15672:15672

# Bash
./scripts/lab-idempotency.sh

# PowerShell
./scripts/lab-idempotency.ps1
```

Ermitteln Sie den Primary-Pod wie in Aufgabe 4. Ergänzen Sie die beiden Platzhalter und prüfen Sie Ihre Prognose.

EVIDENZ

```
kubect1 --context k3d-teko-k8s -n food-delivery exec <PRIMARY> -- psql -d delivery -Atc "SELECT count(*) FROM processed_events WHERE event_id='<EVENT_ID>';"
kubect1 --context k3d-teko-k8s -n food-delivery exec <PRIMARY> -- psql -d delivery -Atc "SELECT count(*) FROM orders WHERE id='<ORDER_ID>';"
```

Sichtbares Resultat: Beide Abfragen liefern 1, obwohl RabbitMQ dieselbe Nachricht zweimal zugestellt hat.

Warum muss processed_events in derselben Transaktion wie die Projektion geschrieben werden?

processed_events und Order zusammen committen, alles oder nichts

sonst bei Crash dazwischen Event erledigt ohne Wirkung oder Order doppelt

6

Failover beobachten und beurteilen

Nicht nur zwei grüne Pods, sondern Rollenwechsel messen.

Wichtig: Der folgende Force-Delete simuliert einen harten Ausfall und ist ausschliesslich für den lokalen k3d-Kurscluster gedacht.

Erfassen Sie zuerst Primary-Pod und -rw-Endpoint. Starten Sie in einem zweiten Terminal die Beobachtung.

BEOBSACHTUNG

```
kubectl --context k3d-teko-k8s -n food-delivery get pods -L cnpg.io/instanceRole -w

# Separates Terminal
kubectl --context k3d-teko-k8s -n food-delivery get endpointslice -l kubernetes.io/service-name=food-delivery-db-rw -w
```

Vorher: Primary-Pod, Node und -rw-Endpoint:

food-delivery-db-1 auf k3d-teko-k8s-server-0, -rw zeigt auf db-1

HARTEN AUSFALL AUSLÖSEN

```
kubectl --context k3d-teko-k8s -n food-delivery delete pod <PRIMARY-POD> --grace-period=0 --force
```

Stoppen Sie die Zeit bis ein anderer Pod die Rolle primary trägt und der -rw-Endpoint gewechselt hat. Prüfen Sie danach Dashboard und Bestellung aus Aufgabe 4.

Messung	Ihr Resultat
Rollenwechsel nach ca.	ca. 15-30 s
Neuer Primary / Node	db-2 auf agent-0
Bestellung noch vorhanden?	ja, noch delivered

Sichtbares Resultat: Der Standby wird Primary, -rw folgt automatisch und das Dashboard bleibt fachlich konsistent.

Kurzes Fazit: Was beweist der Test – und was beweist er noch nicht (Backup/RPO)?

Failover geht automatisch, -rw folgt dem neuen Primary

Backup und RPO sind damit nicht getestet

läuft alles auf meinem Mac, also kein echtes HA